AIVA

Software Requirements Specification

Air-gapped AI candidate evaluation and interview automation platform

Document version: 1.0 | Date: 2026-08-26

Status: Milestones 0–11 delivered and CI-verified; Milestone 12 (production hardening) remaining

Prepared from the current, verified state of the codebase and its governance docs (docs/PLAN.md, docs/DECISIONS.md, docs/RUNBOOK.md, docs/MODEL_CARD.md)

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of AIVA, an on-premise, air-gapped AI-assisted candidate evaluation and interview automation platform. It is written against the system as actually built and verified, not as a forward-looking plan: every requirement below traces to delivered, tested code.

1.2 Scope

AIVA covers the full hiring workflow from job requisition through resume ingestion, deterministic + AI-assisted scoring, candidate questionnaires, interview scheduling, a live adaptive video/audio interview with a sandboxed live-coding exercise, a retrieval-grounded candidate FAQ, a cross-signal evaluation report with PDF/Excel export, and a compliance/operations layer (dashboard, blind screening, scoring-audit, interview integrity signals, reusable questionnaire kits, and GDPR/CCPA DSAR export and erasure). It explicitly excludes, by design, any runtime call to a third-party cloud AI service — every model-backed capability is served locally or behind a stable interface with a deterministic mock standing in until GPU hardware is deployed.

1.3 Definitions, Acronyms, Abbreviations

Term	Meaning
RLS	Postgres Row-Level Security, FORCE-enabled on every org-scoped table
RAG	Retrieval-Augmented Generation — answers grounded only in retrieved documents
DSAR	Data Subject Access Request (GDPR Art. 15/17, CCPA) — export or erasure of a person's data
ADR	Architecture Decision Record (docs/DECISIONS.md) — 23 recorded to date
JWT	JSON Web Token, used for staff access/refresh tokens
MFA	Multi-Factor Authentication (TOTP, RFC 6238)
pgvector	Postgres extension providing native vector columns and similarity search
Air-gapped	No runtime network call leaves the deployment boundary; enforced by static scan and default-deny network policy

1.4 References

- docs/PLAN.md — milestone-by-milestone build and verification ledger
- docs/DECISIONS.md — 23 architecture decision records (ADR-001 through ADR-023)
- docs/RUNBOOK.md — day-2 operations, service ports, test credentials
- docs/MODEL_CARD.md — AI model inventory, licensing, and deployment status
- README.md — developer-facing build and verification narrative

2. Overall Description

2.1 Product Perspective

AIVA is a self-contained monorepo deployed via Docker Compose in development and targeting Kubernetes (Helm chart, M12) in production. It has no dependency on any external SaaS: Postgres (with pgvector + pg_trgm), Redis, and MinIO are the only infrastructure dependencies, all self-hosted. AI capabilities (LLM judgement, speech-to-text, text-to-speech, embeddings) are served by an internal `ai-gateway` microservice behind a stable contract; a deterministic mock backend satisfies every contract today so the full system is provable in CI without GPU hardware, and the real model swap is a backend substitution, not an interface change.

2.2 Product Functions (summary)

- Organization, department, requisition, and staff-user management with six-role RBAC
- Resume ingestion (PDF/DOCX/TXT) with span-preserving field extraction and deterministic + AI-assisted weighted scoring
- Candidate questionnaires with single-use tokenized invites and resumable autosave
- DST-correct interview slot generation and local .ics generation (no external calendar API)
- Consent-gated, device-pre-checked, adaptive live interview with STT/TTS and a real-time HUD
- Sandboxed live-coding workspace: isolated code execution, autosaved editor, shared whiteboard, task discussion
- Retrieval-grounded candidate FAQ (pgvector cosine similarity + cited LLM answer)
- Cross-signal evaluation engine (resume + questionnaire + interview + coding) with PDF/Excel export
- Recruiter dashboard, blind-screening PII redaction, scoring-consistency audit, browser-based interview integrity signals, reusable questionnaire kits
- GDPR/CCPA DSAR export and erasure tooling
- Hash-chained, tamper-evident audit log covering every state-changing action

2.3 User Classes and Characteristics

Role	Characteristics / access
Admin	Full org control; the only role permitted to run DSAR export/erasure
Hiring Manager	Requisition, questionnaire, scoring-profile, and evaluation authority
Recruiter	Day-to-day pipeline operation: upload, score, schedule, interview, evaluate
Interviewer	Read access to sessions, transcripts, and workspace for assigned interviews
Auditor	Read-only access, including the audit-log verification endpoint
Candidate	No account; access is solely via single-use, time-limited, SHA-256-hashed tokens

2.4 Operating Environment

- Backend: Python 3.11, FastAPI, SQLAlchemy 2.0 (async), Postgres 16 + pgvector/pg_trgm, Redis 7, MinIO

- Frontend: React 18, TypeScript (strict), Vite, Tailwind (token-driven design system)
- Containerization: Docker / Docker Compose (dev), Kubernetes + Helm (production target, M12)
- AI serving: internal ai-gateway microservice; sandbox-runner microservice for candidate code execution

2.5 Design and Implementation Constraints

- No source file may contain a literal external URL — enforced by an ESLint rule and a repository-wide scan script against an explicit allowlist
- All runtime configuration arrives via validated environment variables; missing/malformed values fail boot (fail-closed)
- Every persisted AI judgement must carry a rationale, a confidence score, and at least one cited evidence span (constraint 8.1)
- Every org-scoped table is Postgres RLS FORCE-enabled; the runtime database role (aiva_app) never has more privilege than the access pattern requires

2.6 Assumptions and Dependencies

The system assumes GPU hardware is not yet available for the LLM/STT/TTS/embedding workloads; every such capability is therefore built against a stable interface with a deterministic mock backend, and the real-model swap is documented as a deployment-time task (docs/MODEL_CARD.md), not a code change.

3. System Features / Functional Requirements

3.1 Authentication, RBAC, and Audit

1. FR-1.1: The system shall hash passwords with Argon2id and never store plaintext.
2. FR-1.2: The system shall issue short-lived JWT access tokens and rotating refresh tokens, detecting and revoking an entire token family on reuse of a rotated refresh token.
3. FR-1.3: The system shall support TOTP-based MFA enrollment and activation, and reject password-only login once MFA is active for that account.
4. FR-1.4: The system shall enforce a six-role permission matrix (Admin, Hiring Manager, Recruiter, Interviewer, Auditor, Candidate) per endpoint.
5. FR-1.5: The system shall record every state-changing action to an append-only, hash-chained audit log, and expose an endpoint to verify chain integrity.

3.2 Organization and Requisition Management

1. FR-2.1: The system shall support self-service organization registration with an initial admin user.
2. FR-2.2: The system shall support department and requisition CRUD, scoped to the caller's organization via RLS.

3.3 Resume Ingestion, Extraction, and Scoring

1. FR-3.1: The system shall ingest PDF, DOCX, and TXT resumes up to 10MB, rejecting duplicate content by hash.
2. FR-3.2: The system shall extract structured fields (name, email, phone, LinkedIn, skills, years of experience) with page number, character offset, and literal source quote for every field.

3. FR-3.3: The system shall compute a weighted, versioned score (0–100) across seven dimensions, banding into auto_reject / hold / shortlist / highly_recommended, and shall produce a byte-identical run fingerprint for identical resume+profile inputs.
4. FR-3.4: AI-assisted dimension judgements shall each carry a rationale, a confidence score, and at least one cited evidence span id.

3.4 Questionnaires

1. FR-4.1: The system shall support typed, validated questionnaire authoring (multiple-choice, yes/no, rating, short/long text, file upload) with required-field rules.
2. FR-4.2: The system shall issue single-use, SHA-256-hashed candidate invite tokens, shown in raw form exactly once at creation.
3. FR-4.3: The system shall support resumable autosave of candidate answers with a retained history, and enforce required-answer completeness on submit.
4. FR-4.4: The system shall support cloning a questionnaire onto a different requisition ("kit" reuse).

3.5 Scheduling

1. FR-5.1: The system shall generate interview slots from recruiter-defined availability rules (working hours, duration, buffer, weekend exclusion, blackout dates), correctly handling DST transitions (spring-forward gaps absent, no duplicate fall-back wall-clock starts).
2. FR-5.2: The system shall generate RFC-5545-compliant .ics calendar invites locally, with no external calendar API call.
3. FR-5.3: The system shall reject double-booking a slot.

3.6 Live Interview Sessions

1. FR-6.1: The system shall gate every interview through an ordered, fail-closed lifecycle: pending_consent → consent_granted → precheck_passed → active → completed/declined/aborted.
2. FR-6.2: The system shall require the candidate to accept the exact current consent statement version before any device pre-check or question is served.
3. FR-6.3: The system shall validate a device pre-check report (camera, microphone, speaker, connection) against a versioned suite before allowing the interview to start.
4. FR-6.4: The adaptive question engine shall derive its plan deterministically from JD-vs-resume gaps and replay identically given the same transcript, with no LLM involvement in control flow.
5. FR-6.5: The system shall transcribe spoken answers via the AI gateway's STT contract and persist model/confidence attribution per turn.

3.7 Live-Coding Workspace

1. FR-7.1: The system shall execute candidate-submitted Python/JavaScript under process-level isolation: a distinct, never-shared unprivileged OS account per concurrent execution, POSIX resource limits, an isolated network namespace (no egress), an isolated PID namespace, an ephemeral per-run filesystem, and a hard wall-clock timeout.
2. FR-7.2: The system shall autosave the candidate's code editor state and persist every execution's stdout/stderr/exit code/duration.
3. FR-7.3: The system shall provide a shared whiteboard and a threaded discussion, both visible to candidate and interviewer in near-real-time via polling.

4. FR-7.4: The system shall expose a stable screen-share interface that reports 501 Not Implemented rather than a false success, pending WebRTC/LiveKit infrastructure.

3.8 RAG FAQ and Evaluation Engine

1. FR-8.1: The system shall answer candidate questions using only retrieved FAQ documents (pgvector cosine-similarity search), citing the documents used, and shall state plainly when no relevant document exists rather than fabricating an answer.
2. FR-8.2: The system shall deterministically aggregate resume score, questionnaire completion, interview completeness, and coding-task pass rate into one weighted overall score and verdict, renormalizing over whichever signals are actually available.
3. FR-8.3: The system shall render a persisted evaluation report to PDF and Excel on demand, faithful to exactly what was computed and stored.

3.9 Dashboard, Blind Screening, Scoring Audit, Integrity Signals

1. FR-9.1: The system shall provide an organization-wide pipeline dashboard (requisition, scoring-verdict, interview-status, questionnaire-submission, and coding-pass-rate aggregates) containing no candidate-identifying data.
2. FR-9.2: The system shall support a blind-screening mode that redacts name/email/phone/LinkedIn values from the resume list and detail views while preserving skill/experience signal.
3. FR-9.3: The system shall audit its own scoring pipeline for verdict drift across identical re-scored inputs, missing evidence citations, and suspiciously narrow score distributions.
4. FR-9.4: The system shall record browser-reported tab-blur, tab-focus, visibility, and fullscreen-exit events during an active interview as integrity signals, without requiring any ML model.

3.10 Data Subject Access Requests (DSAR)

1. FR-10.1: The system shall, for an admin-role user only, export every record across every subsystem tied to a given candidate email within the caller's organization.
2. FR-10.2: The system shall support erasure of a candidate's personally identifying data by overwriting the specific PII-bearing columns in place — never deleting rows — preserving row counts and non-PII content for audit and statistical integrity.
3. FR-10.3: The system shall record every DSAR export or erasure to the audit log, keyed by a SHA-256 hash of the candidate's email rather than the email itself.

4. External Interface Requirements

4.1 User Interfaces

- apps/web-recruiter — staff console (login, pipeline, resume detail + evaluation, sessions, interview session detail, dashboard)
- apps/web-candidate — token-gated candidate journey (join, consent, device pre-check, live interview HUD with Workspace/FAQ tabs)

4.2 API Interfaces

A single versioned REST/JSON API (FastAPI, OpenAPI schema at `/openapi.json`) is the only interface between the frontends and the backend; the AI gateway and sandbox runner each expose their own internal REST contracts consumed only by the main API, never directly by a browser.

4.3 Hardware Interfaces

None required beyond standard server/container compute for the current mock-backed AI capabilities. GPU hardware is a deployment-time addition for the real LLM/STT/TTS/embedding backends (docs/MODEL_CARD.md).

4.4 Communications Interfaces

All inter-service communication stays within the deployment's internal network (Docker Compose network in development; intended Kubernetes cluster network in production). No runtime call ever crosses the deployment boundary to a third-party service.

5. Non-Functional Requirements

5.1 Security

- NFR-S1: Every organization-scoped table shall enforce Postgres Row-Level Security with FORCE enabled, verified in CI by a two-organization isolation test.
- NFR-S2: Code execution for the live-coding workspace shall never share an OS user identity between two concurrently executing candidates (verified directly by a uid-pool-exhaustion test).
- NFR-S3: The system shall undergo a security review of each milestone's diff before that milestone is considered complete; findings shall be fixed, not merely logged.

5.2 Air-Gap / Network Policy

No source file may contain a literal external URL, enforced redundantly by lint rule and a repository-wide scan against an explicit, reviewed allowlist; the enforcement is itself negative-tested in CI (a planted violation is verified to fail the scan).

5.3 Reliability and Determinism

Resume scoring, interview-plan generation, and the mock AI backends are all deterministic: identical inputs are proven (not merely assumed) to produce byte-identical outputs, which is what makes the entire pipeline provable in CI without live model inference.

5.4 Auditability and Compliance

Every state-changing action is recorded to a hash-chained audit log with an integrity-verification endpoint. DSAR export/erasure gives the organization a working, tested path to GDPR Article 15/17 and CCPA compliance for candidate data specifically.

5.5 Testability

The full domain-lifecycle surface (auth, resume, questionnaire, interview, workspace, FAQ, evaluation, and the full M11 set) is proven end-to-end against a live containerized stack in CI, not only via unit tests against mocked dependencies.

6. System Architecture Overview

AIVA is organized as a monorepo of independently deployable services sharing a Postgres database with per-table Row-Level Security as the primary isolation boundary.

Component	Responsibility
apps/api	Primary FastAPI backend — auth, org/requisition, resume/scoring, questionnaires, scheduling, interviews, live-coding workspace, FAQ, evaluation, dashboard, DSAR
apps/web-recruiter	Staff-facing React console
apps/web-candidate	Candidate-facing React journey (token-gated, no login)
services/ai-gateway	Single internal entry point to LLM/STT/TTS/embedding capabilities, mock-backed today
services/sandbox-runner	Isolated execution of candidate-submitted code for the live-coding exercise
packages/ui	Shared design system: tokens, motion primitives, component library
packages/eval	Golden-set evaluation harness for the AI gateway's determinism guarantee

7. Appendix: Data Model Summary

12 Alembic migrations define the schema; every table is either organization-scoped with FORCE row-level security, or a genuinely public lookup with no organization concept. Key entity groups: Organization/Department/User/Requisition; JobDescription/ResumeDocument/ExtractedFieldRow/WeightProfileRow/ScoringRunRow; Questionnaire/QuestionnaireInvite/QuestionnaireResponse; InterviewSlot/InterviewSession/InterviewTurn/InterviewConsent; CodingTask/CodeSnapshot/CodeExecution/WhiteboardStroke/DiscussionMessage; FaqDocument/EvaluationReport; IntegritySignal; AuditEvent.

End of Software Requirements Specification.